

Part A – Theory & Pre-implementation

1) RC4 Overview

RC4 is a symmetric key stream cipher. It generates a pseudo-random keystream which is XORed with the plaintext to produce ciphertext.

- Type of cipher: Stream cipher
- Role of key: Used to initialize and shuffle the state array
- Keystream: A random byte sequence generated using PRGA
- Why symmetric: The same key is used for both encryption and decryption

Encryption and decryption:

Ciphertext = Plaintext $\oplus$ Keystream

Plaintext = Ciphertext $\oplus$ Keystream

2) Key Scheduling Algorithm (KSA)

1. Initialize state array S of size 256 $\rightarrow S[i] = i$
2. Repeat key using key [i % keylen]
3. $j = (j + S[i] + \text{key}[i \% \text{keylen}]) \bmod 256$
4. Swap S[i] and S[j]

Modulo 256 keeps the index within range.

3) Pseudo Random Generation Algorithm (PRGA)

1. Initialize $i = 0, j = 0$
2. $i = (i + 1) \bmod 256$
3. $j = (j + S[i]) \bmod 256$
4. Swap S[i] and S[j]
5. $t = (S[i] + S[j]) \bmod 256$
6. Keystream byte = S[t]
7. XOR with data

4) Encryption & Decryption Process

RC4 uses XOR operation.

Plaintext = A (65)

Keystream = K (75)

Ciphertext = $65 \oplus 75$

Decryption: Ciphertext $\oplus 75 = 65 = A$

Same algorithm is used because $P \oplus K \oplus K = P$.

Part B – Implementation

Functions implemented:

1. ksa(key, keylen, S)
2. prga(S, data, len)
3. rc4(key, keylen, data, len)
4. main() – accepts key and plaintext, performs encryption and decryption.

Sample Input

Enter key: secret

Enter plaintext: hello

Sample Output

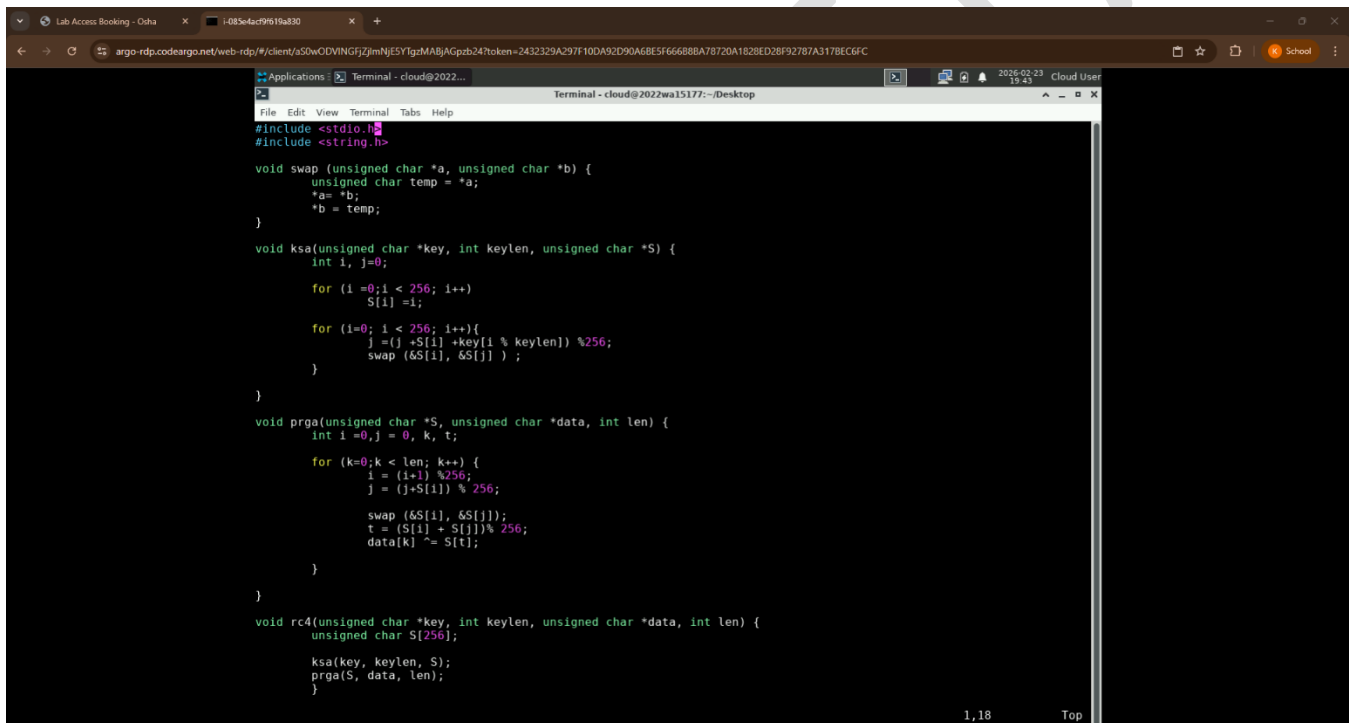
Ciphertext (Hex): XX XX XX XX XX

Decrypted text: hello

Result

The RC4 algorithm was successfully implemented. Encryption and decryption were performed using the same key and the original plaintext was retrieved.

Screenshot of program written in vi editor.



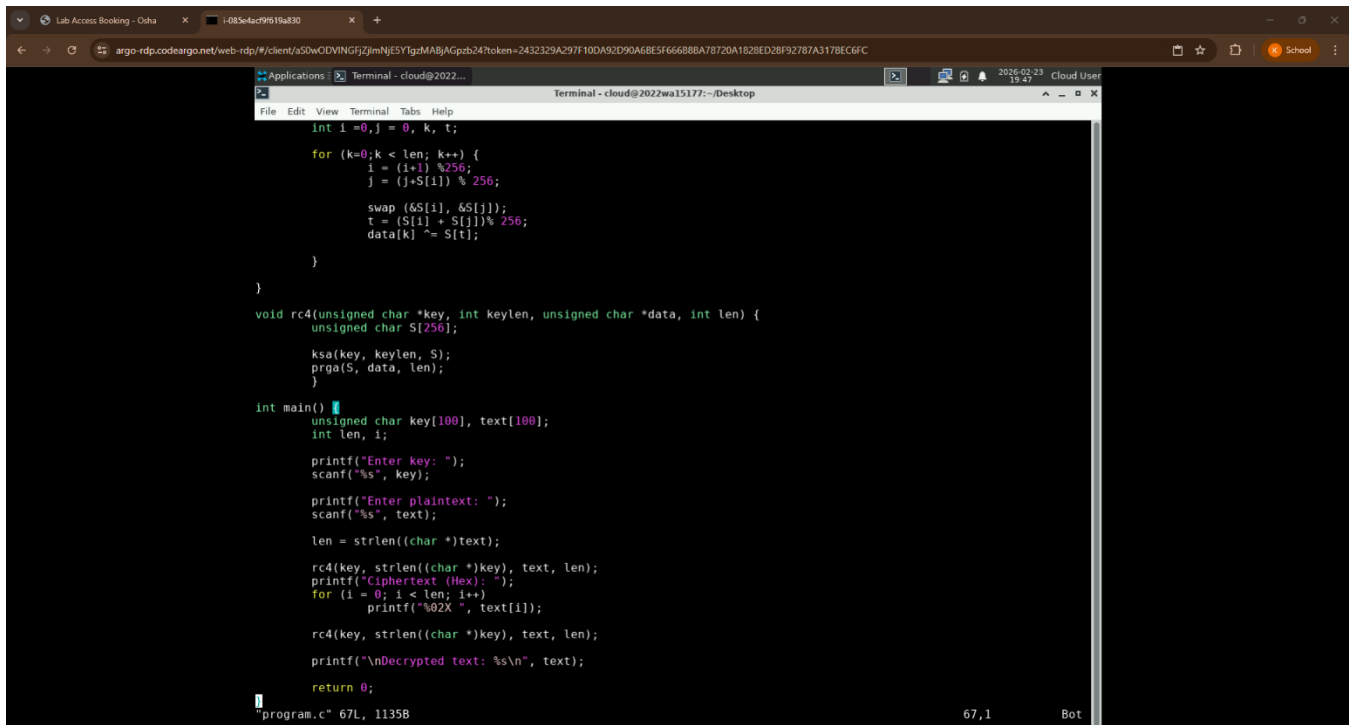
```
#include <stdio.h>
#include <string.h>

void swap(unsigned char *a, unsigned char *b) {
    unsigned char temp = *a;
    *a = *b;
    *b = temp;
}

void ksa(unsigned char *key, int keylen, unsigned char *S) {
    int i, j = 0;
    for (i = 0; i < 256; i++)
        S[i] = i;
    for (i = 0; i < 256; i++) {
        j = (j + S[i] + key[i % keylen]) % 256;
        swap(&S[i], &S[j]);
    }
}

void prga(unsigned char *S, unsigned char *data, int len) {
    int i = 0, j = 0, k, t;
    for (k = 0; k < len; k++) {
        i = (i + 1) % 256;
        j = (j + S[i]) % 256;
        swap(&S[i], &S[j]);
        t = (S[i] + S[j]) % 256;
        data[k] ^= S[t];
    }
}

void rc4(unsigned char *key, int keylen, unsigned char *data, int len) {
    unsigned char S[256];
    ksa(key, keylen, S);
    prga(S, data, len);
}
```



```
int i=0,j = 0, K, t;

for (k=0;k < len; k++) {
    i = (i+1) %256;
    j = (j+S[i]) % 256;

    swap (&S[i], &S[j]);
    t = (S[i] + S[j])% 256;
    data[k] ^= S[t];
}

}

void rc4(unsigned char *key, int keylen, unsigned char *data, int len) {
    unsigned char S[256];

    ksa(key, keylen, S);
    prga(S, data, len);
}

int main()
{
    unsigned char key[100], text[100];
    int len, i;

    printf("Enter key: ");
    scanf("%s", key);

    printf("Enter plaintext: ");
    scanf("%s", text);

    len = strlen((char *)text);

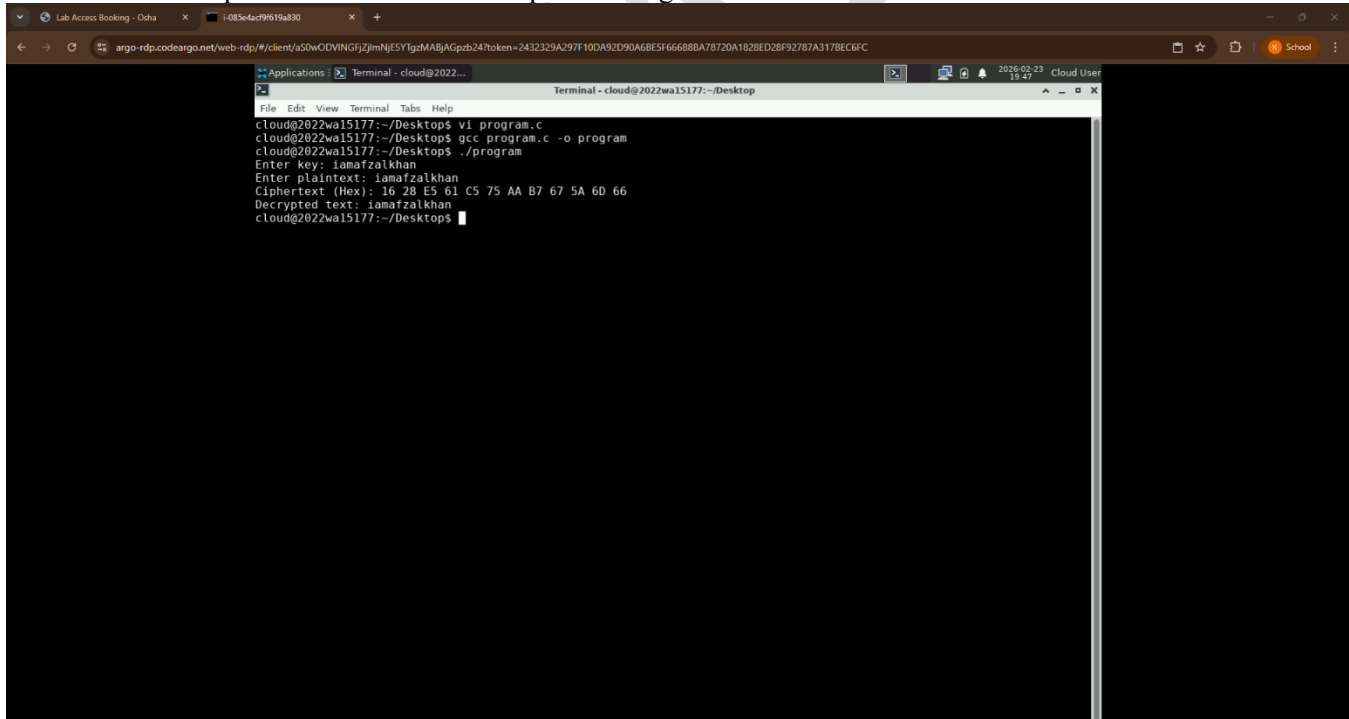
    rc4(key, strlen((char *)key), text, len);
    printf("Ciphertext (Hex): ");
    for (i = 0; i < len; i++)
        printf("%02X ", text[i]);

    rc4(key, strlen((char *)key), text, len);
    printf("\nDecrypted text: %s\n", text);

    return 0;
}

"program.c" 67L, 1135B                                     67,1      Bot
```

Screenshot of compilation and execution output showing username.



```
cloud@2022wa15177:~/Desktop$ vi program.c
cloud@2022wa15177:~/Desktop$ gcc program.c -o program
cloud@2022wa15177:~/Desktop$ ./program
Enter key: iamafzalkhan
Enter plaintext: iamafzalkhan
Ciphertext (Hex): 16 28 E5 61 C5 75 AA B7 67 5A 6D 66
Decrypted text: iamafzalkhan
cloud@2022wa15177:~/Desktop$
```

Observation

- RC4 is fast due to simple operations.
- Same function is used for encryption and decryption.
- XOR operation makes the algorithm reversible.